

Ques

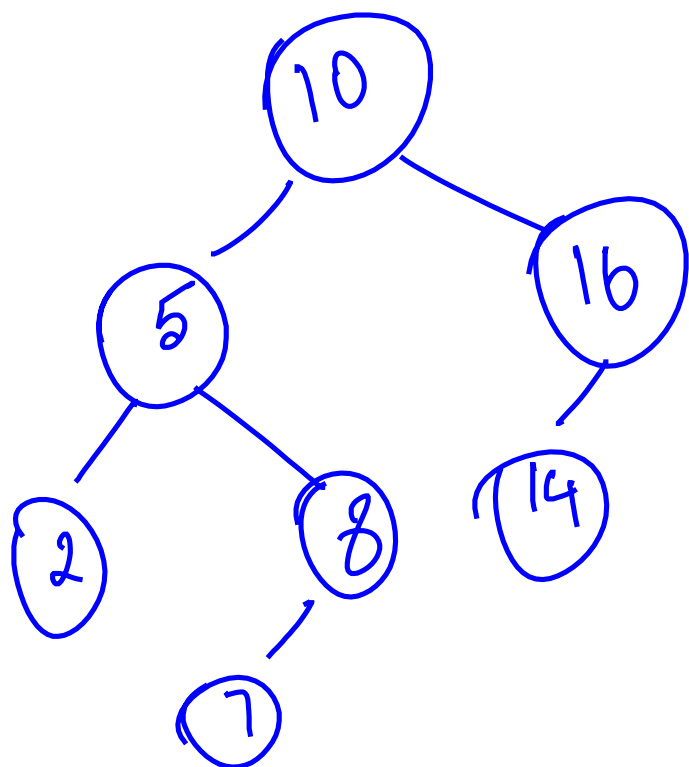
AVL Tree

→ Delete operation

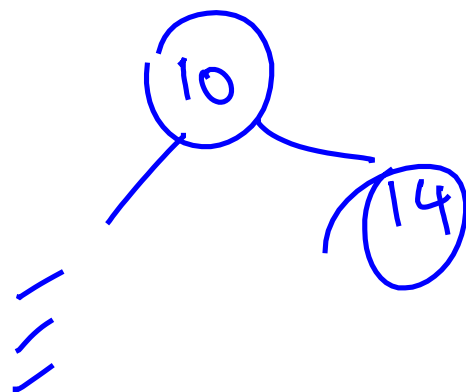
→ Same as BST delete

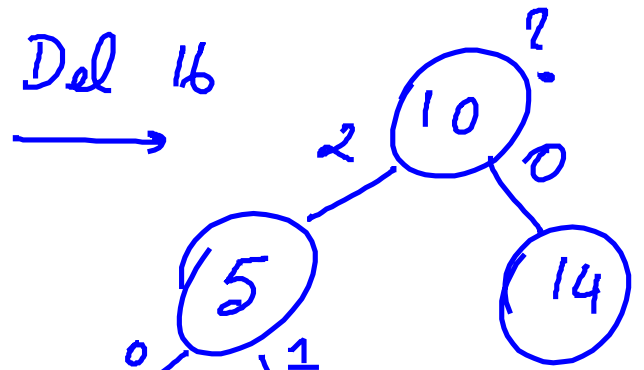
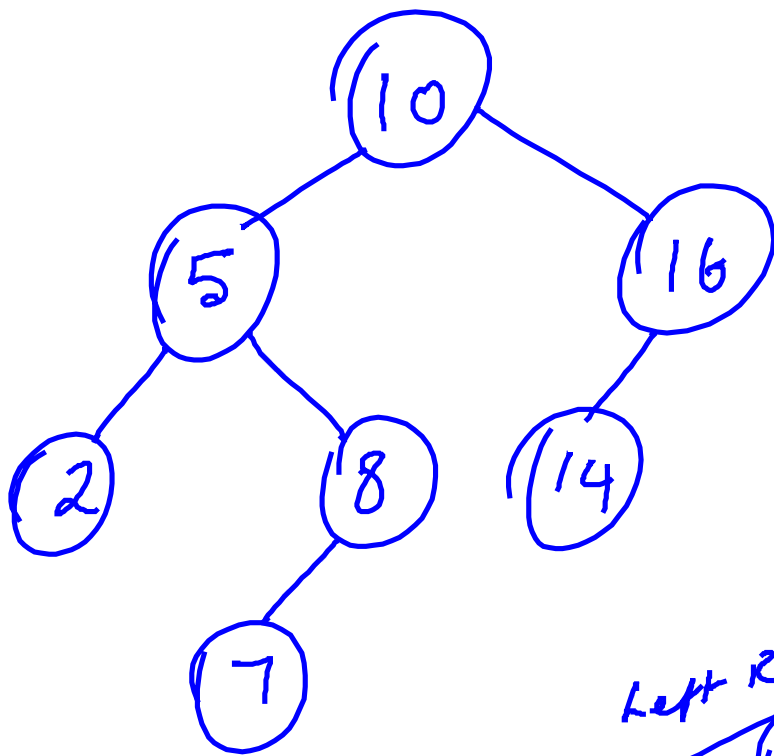
→ After that check for imbalance from bottom to top along the path

→ If a node is imbalanced, check if it is right h



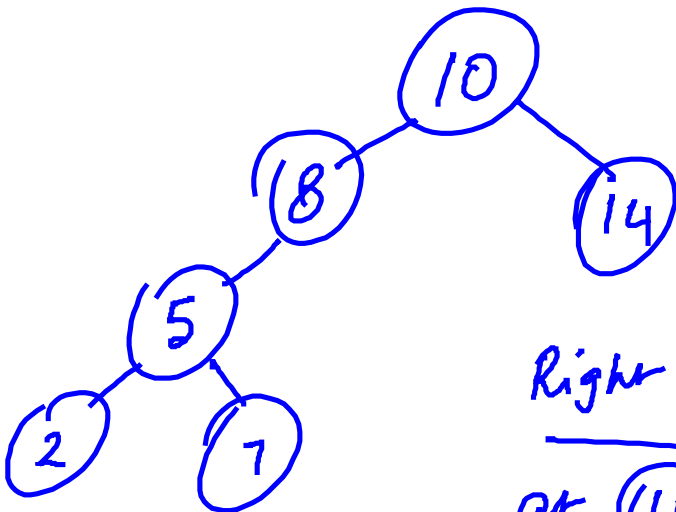
del 16
→



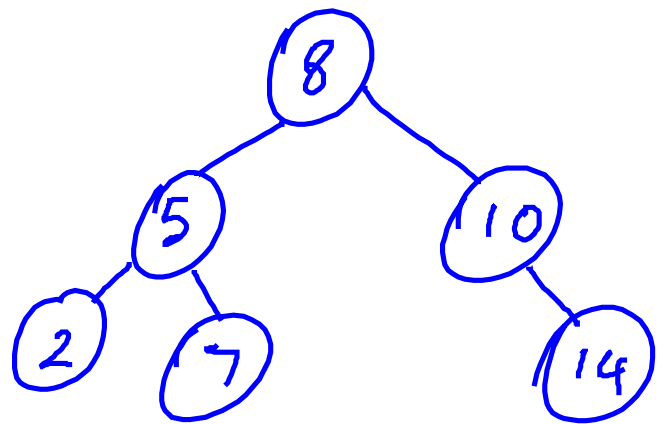


Left Rot.
at 5

?
 ⇒ LSRH
 ⇒ LR Rot.
 ⇒ Left Rot. at 5
 ⇒ Right Rot. at 10

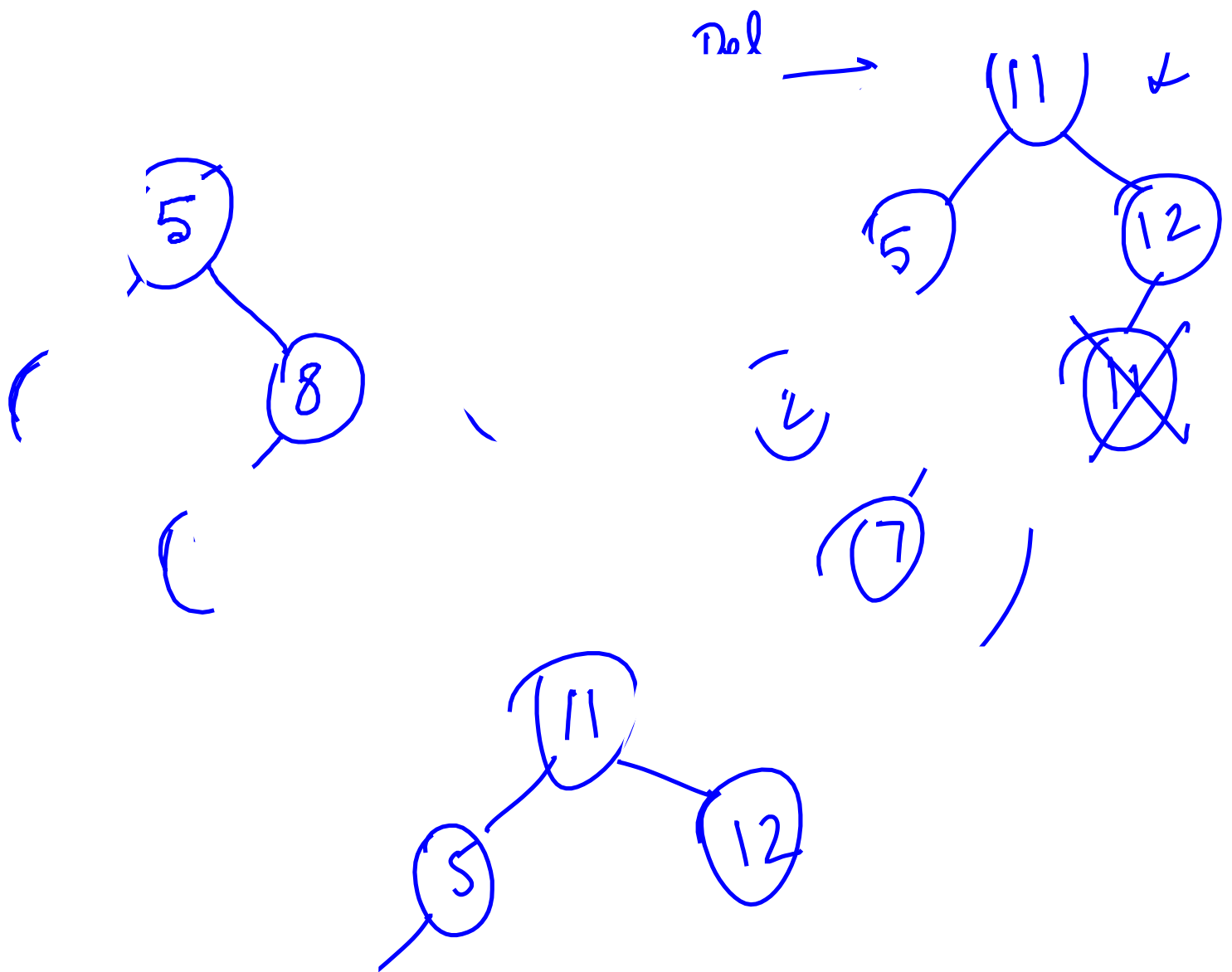


Right Rot.
at 10



44

62



Implementation !

→ Like BST Delete

→ Review ...

struct Node * Delete(struct Node *root,
int x)

{

if (root == NULL)
return (NULL);

if (x < root->val)

root->l = Delete(root->l, x);

else if (x > root->val)

root->r = Delete(root->r, x);

else // root->val == x

{ if (root->l == NULL &&
root->r == NULL)

{ free (root);

return (NULL);

}

if (root->l != NULL &&
root->r == NULL)

{ t = root->l;

free (root);

return (t);

}

if (root → g1 != NULL &&
root → l == N

struct Node * Delete (struct Node *root,
int x)

```
{  
    if (root == NULL)  
        return (NULL);  
  
    if (x < root->val)  
    {  
        root->l = Delete (root->l, x);  
        if (height (root->r) - height (root->l)  
            == 2)  
        {  
            height (root->r->r)  
            = height (root->r->l);  
            root->r = R (root->r->l);  
            return (root);  
        }  
    }  
    else // R.S.  
    {  
        root->r = Delete (root->r, x);  
        if (height (root->l) - height (root->r)  
            == 2)  
        {  
            height (root->l->l)  
            = height (root->l->r);  
            root->l = L (root->l->r);  
            return (root);  
        }  
    }  
}
```


else if ($x > \text{root} \rightarrow \text{val}$)

{ $\text{root} \rightarrow r = \text{Delete}(\text{root} \rightarrow r, x);$

if ($(\text{height}(\text{root} \rightarrow l) - \text{height}(\text{root} \rightarrow r)) == 2$)

{ if ($\text{height}(\text{root} \rightarrow l \rightarrow l) \geq \text{height}(\text{root} \rightarrow l \rightarrow r)$)
// L.S.L.H
return(Right Rotate(root));

else // L.S.R.H
return(LR Rotate(root));

}

}

else // ~~min~~ ~~val~~ --

{ // Same code

}

$\text{root} \rightarrow h = \max(\text{height}(\text{root} \rightarrow l), \text{height}(\text{root} \rightarrow r)) + 1;$

return(root);